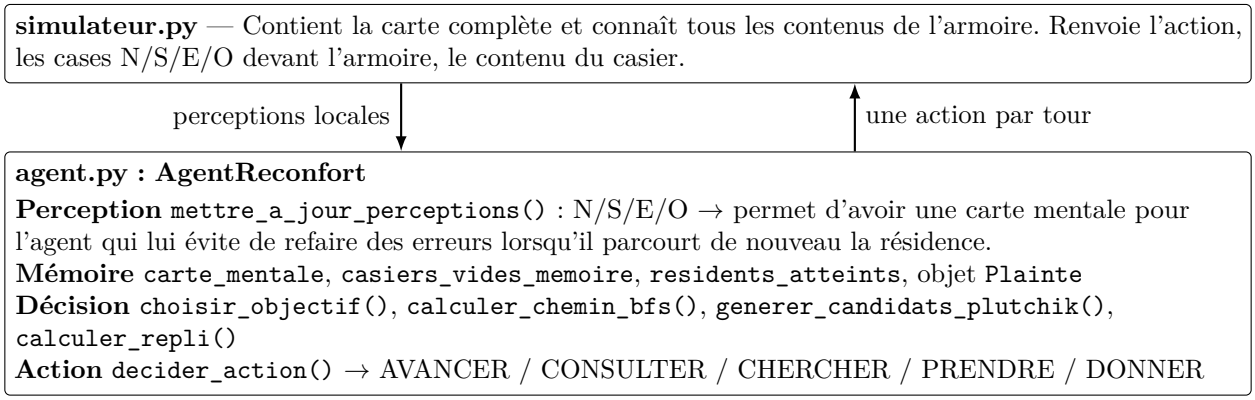
Robot de réconfort — Rapport (PARTIE 1)

HARAUT Kevin - KADDOUR Reyam — Master Informatique, Université de Montpellier

1. Architecture



`reconfort_io.py` + `validation.py` : lecture et contrôle des 4 fichiers *avant* la simulation

Frontière entre la perception globale et locale : Seul le simulateur qui est situé dans le fichier `simulateur.py` est en mesure de lire la grille et le champ `objet` des casiers dans leur globalité. L'agent utilise une carte qui lui est propre dont les seules informations sont sa position, celle des résidents, armoire, et dictionnaire.L'armoire est simplement réduite à des des casiers avec des étiquettes : il ne peut pas tricher, même par erreur, car Le contenu d'un casier ne lui est transmis que lorsqu'il se trouve dessus et qu'il applique l'action adéquate.

Perception et mémoire. Sa perception locale ne lui permet pas directement de prendre une décision. Elle inscrit les quatre cases voisines dans la carte mentale. Nous allons avoir deux durée de vie pour les différentes informations que l'agent va récupérer. La première va être celle qui va durer tout le temps de la simulation comme par exemple la carte mentale, les casiers qui sont connus comme vides, les résidents déjà atteints. Les autres prendront fins après la réalisation ou l'abandon d'une demande comme l'émotion, objet en main ou encore repli. La remise à zéro entre deux demandes est donc garantie par construction.

Décision et action. `choisir_objectif` est une petite machine qui va permettre à l'agent de définir la prochaine étape à réaliser. (`GO_DICO` → `VERIF_RESIDENT` → `GO_ARMOIRE` → `GO_RESIDENT`) ; le BFS et le tri de Plutchik ne font que lire la mémoire. `decider_action` transforme la décision en une action, que le simulateur applique.

2. Stratégie de replanification

Le robot planifie un chemin à l'aide de la méthode BFS sur la carte mentale en supposant libre toute case inconnue (hypothèse optimiste), et en bornant la recherche aux dimensions de la grille. Il garde son plan et ne relance un BFS complet que si le chemin est vide (nouvel objectif) ou si la prochaine case vient d'être perçue comme un mur. La carte mentale ne fait que perdre des cases libres, donc un plan qui ne traverse aucun mur reste un plus court chemin, et comme le robot ne perçoit que ses quatre voisines, seule la case suivante du plan peut se révéler être un mur.

Carte	01	02	03	04	05	**Total**
Pas (identiques dans les deux cas)	210	259	311	302	328	**1 410**
BFS — notre stratégie	25	33	38	37	42	**175**
BFS — un BFS à chaque pas (annexe B)	184	229	263	257	277	**1 210**

Mêmes pas et mêmes livraisons, **environ 7 fois moins de BFS**. En contrepartie, chaque recalcul est complet, en $O(\textit{hauteur} \times \textit{largeur})$, au lieu d'une réparation locale du plan. C'est simple et sûr, mais coûteux sur une très grande carte où les murs se découvrent souvent. La même hypothèse optimiste

fournit la preuve d'inaccessibilité : si le BFS ne trouve aucun chemin alors que l'inconnu est supposé libre, il n'en existe aucun en réalité à condition que la recherche soit finie (échec n° 1). Enfin, entre deux chemins de même longueur, le BFS garde le premier trouvé, car il explore les voisins dans l'ordre N, S, E, O : c'est ce choix qui fixe l'itinéraire exact du robot, sans changer le nombre de pas.

3. Critique de la règle de repli

La règle 6.2 est déterministe (un seul casier possible, vérifiable dans la trace) et reste proche de la demande. Mais la roue de Plutchik range les émotions par ressemblance, pas par ce qui réconforte : les voisines de la tristesse sont la surprise et le dégoût, si bien qu'une personne triste peut recevoir un objet rangé sous « dégoût », alors que la joie, réconfort naturel, est l'émotion opposée et n'est jamais proposée. **Notre avis : la règle est défendable comme règle de test** (simple, sans ambiguïté, vérifiable automatiquement), **mais pas comme règle de soin** : elle peut donner un objet contraire à l'effet recherché.

Règle proposée : une liste de repli choisie par un humain pour chaque émotion, par exemple tristesse → tristesse (autre intensité), confiance, joie ; peur → peur (autre intensité), confiance, sérénité ; colère → colère (autre intensité), sérénité, acceptation. Le robot parcourt la liste dans l'ordre : la règle reste déterministe et vérifiable, mais exprime une intention de réconfort, et un soignant peut l'ajuster sans toucher au code. Il faudrait changer :

— **le fichier d'armoire** : un champ `repli` associant à chaque émotion une liste ordonnée de couples `[émotion, intensité]` ;
— `validation.py` : chaque émotion a une liste, chaque couple désigne un casier existant ;
— **la trace** : `repli` vaudrait `aucun`, `liste_1`, `liste_2`… au lieu de `intensite` / `voisine_1` / `voisine_2` (le rejoueur doit suivre) ;
— `agent.py` : `generer_candidats_plutchik` lirait cette liste au lieu de trier par distance sur la roue ; la fouille et la mémoire des casiers vides ne changent pas.

4. Trois échecs

1. Résident muré au milieu d'une pièce ouverte : le programme ne s'arrêtait pas. Le robot voyait les quatre murs autour du résident avant d'avoir vu l'enceinte de l'appartement. Cause : l'hypothèse optimiste supposait libres aussi les cases hors de l'appartement ($(-1, 5)$, $(-2, 5)$…) ; le BFS explorait un plan infini, alors que la preuve de l'annexe B.6 suppose un graphe fini. Les 5 cartes fournies ne le montraient pas. Correction : borner le BFS à la grille. Le robot abandonne désormais en 21 pas (`resident inaccessible`), les cartes fournies demandent même moins de pas (376 → 328 sur la carte 05), et un test de non-régression couvre ce cas.

2. Dictionnaire ou armoire inaccessible : mauvais motif. Le résident est accessible, mais le dictionnaire est enfermé : le BFS échoue et le robot abandonne avec `resident inaccessible`. Cause : l'abandon sur absence de chemin est écrit pour un seul cas, et l'énoncé ne prévoit que trois motifs. Annoncer `armoire vide` serait pire : ce serait un repli annoncé sans fouille (section 3.3). Piste : des motifs `dictionnaire inaccessible` et `armoire inaccessible` dans le format de trace.

3. Le sélecteur ne retient pas ce qu'il voit en passant. Seuls les casiers visés sont mémorisés comme vides : un casier vide simplement traversé est oublié, et le sélecteur peut faire des allers-retours dans l'armoire (phénomène décrit en annexe C). Cause : la mémoire n'est mise à jour que dans la branche « casier cible atteint ». Piste : mémoriser tout casier vide traversé, et choisir l'itinéraire du sélecteur en tenant compte des candidats suivants.

Coût assumé. Le robot vérifie que le résident est accessible avant de prendre l'objet, car aucune action ne permet de le reposer. Sans cette vérification, les 5 cartes demandent 1 182 pas au lieu de 1 410 : elle coûte **19 %** de pas en plus (de 11 à 31 % selon la carte), pour aucune livraison gagnée ici. C'est le prix d'une garantie : ne jamais rester chargé d'un objet impossible à livrer.